OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

Week-6 (Part-B)

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

lenovo@SAISHASHANK:~$ cd ~

lenovo@SAISHASHANK:~$ mkdir -p Week6

lenovo@SAISHASHANK:~$ cd Week6

lenovo@SAISHASHANK:~/Week6$ nano task1.c

lenovo@SAISHASHANK:~/Week6$ gcc task1.c -o task1

lenovo@SAISHASHANK:~/Week6$ ./task1

Task 1: Command Parser

Enter a command (type exit to quit)

parser> hello world

hellParser output: hello world

Parser validation: PASS

o\ world

hello "parser> Parser output: hello world

Parser validation: PASS

parser> world test"

hellParser output: hello world test

Parser validation: PASS

parser> o 'world test'

hParser output: hello world test

Parser validation: PASS

parser> ello\;world

Parser output: hello;world

Parser validation: PASS

parser> exit

C code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 1024
int parse_command(const char *input, char *output) {
int i = 0, j = 0;
char quote = '\0';
while (input[i] != '\0' && input[i] != '\n') {
char c = input[i];
// Handle escaped characters, including spaces
if (c == '\\') {
if (input[i + 1] == '\0' || input[i + 1] == '\n')
return -1;
output[j++] = input[i + 1];
i += 2;
}
// Handle single and double quotes
else if (c == '\" || c == '"') {
if (quote == '\0')
quote = c;
else if (quote == c)
quote = '\0';
else
output[j++] = c;
i++;
```

```
}
// Preserve characters inside quotes
else {
if (quote == '\0' && c == ';') {
printf("Special symbol detected: ;\n");
}
output[j++] = c;
i++;
}
if (j >= MAX - 1)
return -1;
}
if (quote != '\0')
return -1;
output[j] = '\0';
return 0;
}
int main(void) {
char input[MAX], output[MAX];
printf("Task 1: Command Parser\n");
printf("Enter a command (type exit to quit)\n");
while (1) {
printf("parser> ");
if (fgets(input, sizeof(input), stdin) == NULL)
break;
input[strcspn(input, "\n")] = '\0';
```

```
if (strcmp(input, "exit") == 0)

break;

if (parse_command(input, output) == 0) {

printf("Parser output: %s\n", output);

printf("Parser validation: PASS\n");

} else {

printf("Parser validation: FAIL\n");

printf("Reason: unmatched quote, trailing escape, or input too long\n");

}

}

return 0;

}
```

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
lenovo@SAISHASHANK:~/Week6$ nano task2.c
lenovo@SAISHASHANK:~/Week6$ gcc -Wall -Wextra task2.c -o task2
./task2
Task 2: Process Launcher
Enter commands (type exit to quit)
launcher> pwd
ls -l
date
echo Hello World
Parent PID: 648, Child PID: 649
invalidcommand123
exit/home/lenovo/Week6
Child exited with status: 0
launcher> Parent PID: 648, Child PID: 650
total 44
-rwxr-xr-x 1 lenovo lenovo 16256 Oct 13 03:21 task1
-rw-r--r-- 1 lenovo lenovo  1855 Oct 13 03:20 task1.c
-rwxr-xr-x 1 lenovo lenovo 16600 Oct 13 03:23 task2
-rw-r--r-- 1 lenovo lenovo  1969 Oct 13 03:23 task2.c
Child exited with status: 0
launcher> Parent PID: 648, Child PID: 651
```

```
Tue Oct 13 03:23:54 UTC 2026
Child exited with status: 0
launcher> Parent PID: 648, Child PID: 652
Hello World
Child exited with status: 0
launcher> Parent PID: 648, Child PID: 653
Execution failed: No such file or directory
Child exited with status: 127
launcher> exit
Parent PID: 648, Child PID: 692
Execution failed: No such file or directory
Child exited with status: 127
launcher> exit
Parent process exiting.
lenovo@SAISHASHANK:~/Week6$ # Task 1
gcc -Wall -Wextra task1.c -o task1
./task1

# Task 2
gcc -Wall -Wextra task2.c -o task2
./task2
Task 1: Command Parser
Enter a command (type exit to quit)
parser> exit
Task 2: Process Launcher
Enter commands (type exit to quit)
launcher> exit
Parent process exiting.
```

C code :

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#define MAX 1024

#define MAX_ARGS 64

int main(void) {
```

```
char input[MAX];
printf("Task 2: Process Launcher\n");
printf("Enter commands (type exit to quit)\n");
while (1) {
printf("launcher> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
break;
input[strcspn(input, "\n")] = '\0';
if (strcmp(input, "exit") == 0)
break;
if (input[0] == '\0')
continue;
char *args[MAX_ARGS];
int count = 0;
char *token = strtok(input, " \t");
while (token != NULL && count < MAX_ARGS - 1) {
args[count++] = token;
token = strtok(NULL, " \t");
}
args[count] = NULL;
if (count == 0)
continue;
pid_t pid = fork();
if (pid < 0) {
perror("fork failed");
```

```
continue;
}
if (pid == 0) {
// Child process executes the requested program
execvp(args[0], args);
// Reached only if execvp fails
perror("Execution failed");
_exit(127);
} else {
// Parent waits for the child to finish
int status;
printf("Parent PID: %d, Child PID: %d\n",
(int)getpid(), (int)pid);
if (waitpid(pid, &status, 0) == -1) {
perror("waitpid failed");
continue;
}
if (WIFEXITED(status)) {
printf("Child exited with status: %d\n",
WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
printf("Child terminated by signal: %d\n",
WTERMSIG(status));
}
}
}
```

```
printf("Parent process exiting.\n");

return 0;

}
```